

Team Productivity Application

Collaborative Agile Workspace — Complete MVP Specification

Version: 2.0 Date: July 2026 Status: Final

- React
- Node.js
- PostgreSQL
- JWT Auth
- Role-Based Access
- Sprint Management
- Anonymous Retro

TABLE OF CONTENTS

1	Problem Statement	6	Out of Scope
2	Target Users	7	Success Metrics
3	User Stories	8	Open Questions
4	Core Features	9	Constraints
5	Non-Functional Requirements	—	Must-Have Acceptance Criteria

1 Problem Statement

Small software teams running agile sprints rely on separate tools for task tracking and retrospectives. This separation means retro feedback is disconnected from the work that just happened — retros get scheduled separately, skipped under deadline pressure, or conducted without reference to what the sprint actually produced.

Core problem: When sprint execution and retrospective reflection live in different tools, teams lose the contextual link between *what was done* and *how the team experienced it* — reducing retrospective quality and making continuous improvement harder to sustain.

Who it affects: Project managers and developers on teams of 4–8 people running 1–2 week sprints.

What success looks like: A team completes a sprint, is immediately prompted for structured feedback grounded in that sprint's tasks, and the PM reviews responses in context — all without leaving the application.

Alex**PROJECT MANAGER / TECH LEAD**

Team size	4–8 developers
Cadence	1–2 week sprints
Tools today	Jira; no consistent retro tool
Pain point	Retros happen days later in a different tool, or not at all. No contextual link to sprint work.
Goal	Run the full sprint lifecycle — create, track, close, collect feedback, review — from one workspace.

Priya**DEVELOPER / INDIVIDUAL CONTRIBUTOR**

Team size	4–8 teammates
Cadence	1–2 week sprints
Tools today	Jira for tasks; ad-hoc retros in Miro or Google Forms
Pain point	Retro forms are generic, disconnected from sprint work, and non-anonymous — discouraging honest feedback.
Goal	See sprint tasks on a shared board and submit candid anonymous feedback when a sprint ends.

AUTHENTICATION & ONBOARDING

ID	USER STORY	ACCEPTANCE CRITERIA
US-01	As a PM , I want to register and create a team, so that I can invite developers to a shared workspace.	• Registration succeeds with name, email, password, role=PM • Team created automatically; unique 6-digit alphanumeric invite code generated • Invite code displayed on PM dashboard after registration
US-02	As a Developer , I want to join a team using an invite code, so that I can access the shared sprint board.	• Valid code → user joins team and is redirected to dashboard • Invalid code → error: "Invalid invite code. Please check with your Project Manager." • Second PM registration with same invite code → error: "A Project Manager already exists for this team."
US-03	As any user , I want to log in and log out securely, so that my session is clearly bounded.	• Valid credentials → JWT issued; redirect to dashboard • Invalid credentials → "Invalid email or password." (same message for both fields) • Logout clears JWT from localStorage; redirect to login • Protected route with expired/missing JWT → redirect with "Session expired. Please log in again."

TASK MANAGEMENT

ID	USER STORY	ACCEPTANCE CRITERIA
US-04	As a PM , I want to create tasks in a backlog, so that I have a pool of work to pull into sprints.	• Task created with required title, optional description, optional assignee, priority • Task appears in Backlog with status Backlog • Empty backlog shows: "No tasks in backlog. Create a task to get started."
US-05	As a PM , I want to add backlog tasks to the active sprint, so that the team has a defined scope.	• PM selects tasks and assigns them to active sprint; tasks appear in To Do column • Action disabled if no sprint is active

SPRINT MANAGEMENT

ID	USER STORY	ACCEPTANCE CRITERIA
US-06	As a PM , I want to create a sprint with a name and end date, so that the team has a defined iteration window.	• Sprint requires name and end date (today or future); start date defaults to today • Past end date → "End date must be today or in the future." • Creating when sprint active → "A sprint is already active. Complete the current sprint first."

ID	USER STORY	ACCEPTANCE CRITERIA
		Creating while retro is open → "Close the current retrospective before starting a new sprint."
US-07	As a PM , I want to delete an empty sprint, so that I can correct mistakes at creation.	- Sprint with 0 tasks and Active status: delete permitted after confirmation - Sprint with ≥ 1 task: delete button disabled with tooltip "Remove all tasks from this sprint before deleting."

KANBAN BOARD

ID	USER STORY	ACCEPTANCE CRITERIA
US-08	As any team member , I want to see all sprint tasks on a Kanban board, so that the team has shared visibility into progress.	- Board shows three columns: To Do, In Progress, Done with task count badges - Cards show: title, assignee initials, priority badge - Empty column shows: "No tasks here." - No active sprint → "No active sprint. Create a sprint to get started." (Create Sprint button for PM only)
US-09	As any team member , I want to move task cards between columns, so that the board reflects current work status.	- PM and Developers can drag cards between columns - Status update persists to server immediately - Server failure → card snaps back; toast: "Failed to update task. Please try again." - Board is not draggable when sprint status is Completed

SPRINT COMPLETION

ID	USER STORY	ACCEPTANCE CRITERIA
US-10	As a PM , I want to close a sprint with a confirmed action, so that the iteration ends formally and retro is triggered.	"Complete Sprint" visible to PM only when sprint is Active - Confirmation dialog shows count of incomplete tasks - On confirm: sprint → Completed; incomplete tasks → Backlog; board locks; retro unlocks for all - Dashboard transitions to Sprint Complete state with retro prompt as primary content
US-11	As a PM , I want incomplete tasks to return to backlog on sprint close, so that no planned work is lost.	- All To Do and In Progress tasks reset to Backlog with <code>sprint_id = null</code> on close - Done tasks remain attached to the sprint - Backlog view reflects returned tasks immediately

RETROSPECTIVE FORM

ID	USER STORY	ACCEPTANCE CRITERIA
US-12	As any team member , I want to be prompted to submit retro feedback after sprint close, so that my reflections are captured while context is fresh.	- Dashboard shows Sprint Complete state with banner: "Sprint [name] is complete. Share your feedback." - Done tasks from the sprint displayed above the form as context - Banner persists until user submits or PM closes retro
US-13	As any team member , I want to submit anonymous answers to three structured questions, so that I can share honest feedback without attribution.	- Three required fields: "What went well?", "What didn't go well?", "What should we improve?" - Min 10 chars, max 500 chars per answer; character counter shown - Successful submission → "Your feedback has been submitted. Thank you." - user_id stored server-side only; never returned in any client-facing API response
US-14	As a team member , I want to be prevented from submitting retro more than once, so that results are not skewed.	- Unique constraint on (sprint_id, user_id) enforced at database level - Second attempt → HTTP 409; page shows submitted confirmation state on reload

RETRO RESULTS

ID	USER STORY	ACCEPTANCE CRITERIA
US-15	As a PM , I want to view anonymous responses alongside completed tasks, so that I can review feedback in context of what was delivered.	- PM only; Developer access → HTTP 403: "Access denied." - Left panel: Done tasks (title, assignee initials, priority) - Right panel: responses grouped by question; no names shown - Header: "X of Y team members responded" — PM's own submission counted in both - Results viewable before all members submit
US-16	As a PM , I want to see how many members have submitted, so that I know if the picture is complete.	"X of Y team members responded" shown prominently - Count updates on page refresh - Individual submitter identities never revealed in UI or API response
US-17	As a PM , I want to close the retro to archive the sprint and enable the next one.	"Close Retro" button on results page - On close: retro prompt dismissed for all users; new sprint creation enabled - Results remain permanently accessible from Sprint History - PM cannot create new sprint while retro is open → "Close the current retrospective before starting a new sprint."

SPRINT HISTORY

ID	USER STORY	ACCEPTANCE CRITERIA
US-18	As a PM , I want to access past sprint retro results at any time, so that I can reference previous feedback.	• Sprint History lists completed sprints in reverse chronological order (PM only) • Each row: sprint name, dates, task completion rate (X/Y done), retro response rate • Clicking a sprint opens its Retro Results view in read-only mode

FEATURE	DESCRIPTION	PRIORITY
User Auth & Session	Email/password register and login with JWT. Secure logout. Session expiry handling. Two roles: PM and Developer.	Must Have
Team Creation & Invite	PM creates team on registration. 6-digit alphanumeric invite code. Developers join via code. One PM per team.	Must Have
Backlog & Task Management	PM creates, edits, deletes backlog tasks. Fields: title, description, assignee, priority, status.	Must Have
Sprint Creation & Management	PM creates sprints with name and end date. One active sprint at a time. Sprint deletion before tasks are added.	Must Have
Kanban Board	Three-column board (To Do / In Progress / Done). PM and Developers move cards. Board locks on sprint close.	Must Have
Role-Based Access Control	PM-only routes return HTTP 403 for Developer role. Enforced at API middleware level on every request.	Must Have
Sprint Completion Trigger	PM closes sprint via confirmed action. Incomplete tasks return to backlog. Board locks. Retro unlocks for all. New sprint blocked until retro closed.	Must Have
Anonymous Retro Form	Three fixed open-text questions. One submission per user per sprint. user_id stored for tracking only; never returned in client-facing responses.	Must Have
Retro Results View	PM sees aggregated anonymous responses by question alongside completed task list. Response count shown. PM closes retro to enable next sprint.	Must Have
Sprint History	PM views all completed sprints with completion and response rates. Full retro results permanently accessible.	Should Have
Team Members View	PM sees all team members with name, role, join date. Invite code visible and copyable.	Should Have
Task Assignment & Priority	Tasks carry optional assignee and priority (Low/Medium/High). Shown as badges on Kanban cards.	Should Have
Sprint Goal Display	Optional one-line sprint goal shown at top of Kanban board.	Nice to Have
Retro Response Rate Indicator	Dashboard shows live count of how many members have submitted retro after sprint close.	Nice to Have

SECURITY

- Passwords hashed with bcrypt, minimum 10 salt rounds
- JWT signed with HS256, 24-hour expiry; no refresh token in MVP
- Login rate-limited: HTTP 429 after 10 failed attempts per IP within 5 minutes
- All routes `/api/auth/register` and `/api/auth/login` require valid `Authorization: Bearer` header except JWT in `Bearer`
- Role enforcement applied in API middleware; PM-only endpoints return `{ "error": "Access denied." }` HTTP 403 with body
- `user_id` excluded from retro results API at `COUNT(DISTINCT user_id)` aggregate is permitted; no individual `user_id` in response query level — only

INPUT VALIDATION RULES

FIELD	RULE
Email	Valid email format; max 255 characters; unique across system
Password	Min 8 characters; max 128 characters
Name	1–100 characters; required
Invite code	Exactly 6 alphanumeric characters
Task title	1–100 characters; required
Task description	Max 1000 characters; optional
Sprint name	1–100 characters; required
Sprint end date	ISO 8601 date; must be today or a future date
Retro answers	Min 10 characters; max 500 characters; all three required

API ERROR RESPONSE FORMAT

All API errors return a consistent JSON envelope. `field` is included only for validation errors.

```
{ "error": "Human-readable message.", "field": "fieldName" }
```

HTTP status codes:

- 400 Validation error** — missing or invalid field
- 401 Unauthenticated** — missing or invalid JWT
- 403 Forbidden** — valid JWT but insufficient role
- 404 Not found** — resource does not exist


```
409 Conflict           – duplicate submission, sprint already active
429 Rate limited       – too many login attempts
500 Server error       – unexpected failure
```

PERFORMANCE

- Dashboard, Kanban board, and Retro Results pages load in under 3 seconds on standard broadband
- API responses for list endpoints return in under 1 second for teams up to 10 members and backlogs up to 100 tasks

DATA INTEGRITY

- Sprint closure is atomic — all state changes execute in a single database transaction; partial state is not possible
- Sprint closure is irreversible at database level — no `sprints.status` from `completed` back `active`; `UPDATE` permitted on `sprints` to `completed`
- Retro submissions have a unique constraint `(sprint_id, user_id)`; duplicates rejected at database level

LIST LIMITS (MVP)

- Backlog: first 100 tasks ordered by creation date descending; no pagination UI
- Sprint History: first 50 sprints ordered by end date descending
- Retro results: all submissions for a sprint (team size bounded at <20)

ANONYMITY MODEL (EXPLICIT)

DESIGN DECISION — SOFT ANONYMITY

This application implements **soft anonymity**. `user_id` is stored in `retro_submissions` for submission tracking purposes. It is never included in any API response sent to a client. The PM sees response content and an aggregate count of submitters, but not individual identities. In teams of 4–8 people, responses may be inferable from context, writing style, or timing. This is a known tradeoff between usability and strict anonymity, documented here as an explicit product decision.

BROWSER SUPPORT

Latest two versions of Chrome, Firefox, and Microsoft Edge. Safari not tested. No IE11 support.

Any feature not listed in Section 4 and not excluded below should be treated as out of scope by default and flagged before building.

USER & ACCOUNT MANAGEMENT

- ✗ Password reset or account recovery
- ✗ User profile editing (name, email, password)
- ✗ PM role transfer to another member
- ✗ Team deletion
- ✗ Email verification on registration
- ✗ Account deletion or data erasure
- ✗ Removing or deactivating team members
- ✗ Multiple teams per user account

TASK & SPRINT MANAGEMENT

- ✗ Sprint editing after creation
- ✗ Multiple concurrent active sprints
- ✗ Task due dates
- ✗ File or image attachments on tasks
- ✗ Task reordering within a column
- ✗ Burndown charts or velocity tracking
- ✗ Sprint reopening after close
- ✗ Subtasks or task dependencies
- ✗ Task comments or activity log
- ✗ Task search, filtering, or sorting
- ✗ Story points or effort estimation
- ✗ Task labels or custom tags

RETROSPECTIVE

- ✗ Custom or configurable retro questions
- ✗ Voting or dot-voting on retro responses
- ✗ Action item tracking from retro
- ✗ Developer access to retro results
- ✗ Retro auto-close (time-based expiry)
- ✗ Alternative retro formats (4Ls, Start/Stop/Continue)
- ✗ Grouping or theming retro responses
- ✗ Retro editing or deletion after submission
- ✗ Sentiment analysis or auto-summarisation

NOTIFICATIONS & INTEGRATIONS

- ✗ Email notifications of any kind
- ✗ Browser push notifications
- ✗ Jira, Linear, or GitHub Issues import
- ✗ Slack or Microsoft Teams integration
- ✗ Outbound webhooks or public API
- ✗ In-app notification centre or badge
- ✗ Retro submission reminders
- ✗ GitHub / GitLab commit and PR linking
- ✗ Google, GitHub, or any OAuth / SSO login
- ✗ Calendar integrations for sprint dates

REAL-TIME, PLATFORM & COMPLIANCE

- ✗ WebSocket or live board synchronisation
- ✗ Offline mode or local-first data
- ✗ Mobile-responsive design
- ✗ Internationalisation or localisation
- ✗ Two-factor authentication
- ✗ GDPR data export or right-to-erasure tooling
- ✗ Presence indicators or collaborative cursors
- ✗ iOS or Android mobile application
- ✗ WCAG accessibility compliance
- ✗ Dark mode
- ✗ Audit logging of user actions
- ✗ Cross-sprint trend analysis or data export

METRIC	DEFINITION	TARGET
Sprint close rate	% of created sprints formally closed via "Complete Sprint"	≥ 90%
Retro completion rate	% of closed sprints with at least one retro response submitted	≥ 80%
Retro participation rate	Average % of team members who submit retro per sprint	≥ 75% per sprint
Retro submission window	% of sprints where first retro submission occurs within 48 hours of sprint close	≥ 70%
End-to-end demo completion	% of users who complete the full flow (register → sprint → close → retro → results) at least once	100%
Task completion rate	Average % of sprint tasks reaching Done status before close	Tracked; no v1 target

ID	QUESTION	WHY IT MATTERS
OQ-01	Should there be a time limit on retro submissions — e.g. retro auto-closes 7 days after sprint close if PM has not done so manually?	Without a limit the retro prompt persists indefinitely. PM may forget to close it, permanently blocking the next sprint.
OQ-02	After submitting the retro, should a Developer be able to view the list of Done tasks from the completed sprint?	Developers currently have no post-sprint visibility. Showing the task list (not responses) may support transparency without compromising anonymity.
OQ-03	Should a Developer who joins the team after a sprint closes be counted in the "Y" of "X of Y responded"?	A late-joining developer cannot retroactively submit. Including them in Y skews participation count downward without cause.

CONSTRAINT	DETAIL
Technology stack	Frontend: React. Backend: Node.js (Express). Database: PostgreSQL. No alternatives permitted for MVP.
Single active sprint	Exactly one sprint may be Active per team at any time. Enforced at both API and database level.
Retro blocks next sprint	A new sprint cannot be created while the previous sprint's retro is open. PM must close the retro first. This is a deliberate workflow gate.
No real-time sync	REST request-response only. Concurrent edits are last-write-wins. Board state is accurate as of the last page load. No WebSockets.
Fixed retro questions	Three retrospective questions are hardcoded. No configuration interface provided or planned for v1.
One team per user	A user account is permanently bound to the team they register into. No team-switching or multi-membership.
Soft anonymity	Responses are anonymous in the UI, but user identity is stored server-side for submission tracking. Documented design decision; see Section 5.
No email infrastructure	No SMTP server or transactional email service. Invite codes are shared manually by the PM outside the application.
Assignment scope	Feature scope bounded by student project deadline. Production hardening (rate limiting beyond login, audit logs, GDPR tooling) explicitly deferred.